

# INSTITUTO TECNOLÓGICO DE LAS AMÉRICAS (ITLA)

Escuela de Desarrollo de Software

## DOCUMENTO TÉCNICO DE ENTREGA

Desarrollo y Consumo de la Capa de Presentación Web (SIGEBI)

---

|                            |                                                                                         |
|----------------------------|-----------------------------------------------------------------------------------------|
| <b>Asignatura:</b>         | Programación 2                                                                          |
| <b>Profesor:</b>           | Francis Ramírez                                                                         |
| <b>Estudiante:</b>         | Franklyn Enmanuel Santana Rodríguez                                                     |
| <b>Matrícula:</b>          | 2025-2089                                                                               |
| <b>Fecha de Entrega:</b>   | 3 de Agosto de 2026                                                                     |
| <b>Repositorio GitHub:</b> | <a href="https://github.com/Osiris3939/SIGEBI">https://github.com/Osiris3939/SIGEBI</a> |

---

Santo Domingo, República Dominicana

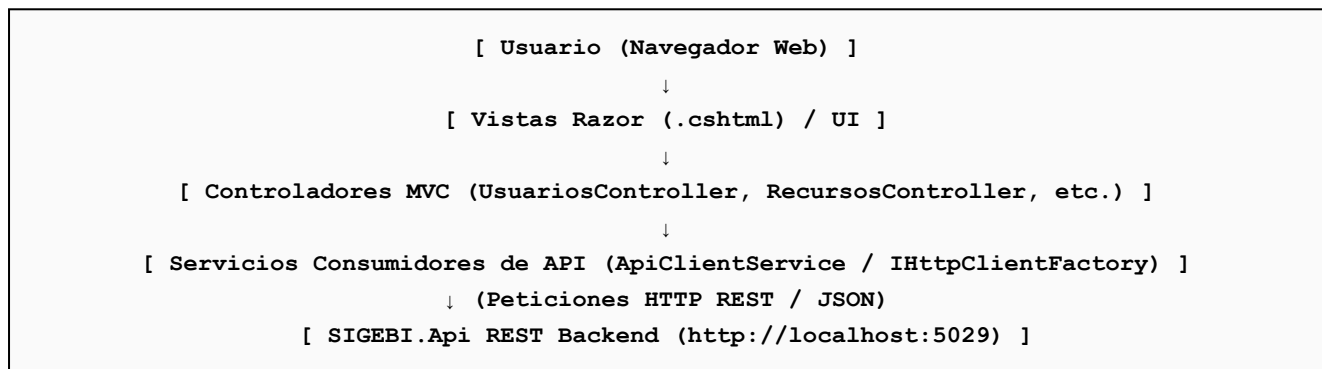
## 1. ARQUITECTURA DE LA SOLUCIÓN

El sistema **SIGEBI** (Sistema de Gestión de Bibliotecas Institucional) implementa una arquitectura N-Tier desacoplada bajo los principios de **Arquitectura Limpia (Clean Architecture)** y **Diseño Orientado a Servicios (SOA)**. La capa de presentación se encuentra estrictamente separada del backend mediante contratos de servicio HTTP RESTful.

El backend (**SIGEBI.Api**) se ejecuta de manera independiente en `http://localhost:5029` utilizando ASP.NET Core Web API y exponiendo los recursos de negocio. La capa de presentación (**SIGEBI.Web**) se ejecuta en `http://localhost:5023` sobre ASP.NET Core MVC y consume dichos endpoints exclusivamente mediante servicios de API desacoplados sin acceso directo a la persistencia.

## 2. ARQUITECTURA LÓGICA DE LA CAPA DE PRESENTACIÓN (OBLIGATORIO)

### Diagrama de Flujo de Arquitectura Lógica Interna



### Explicación de Componentes

- **Controladores MVC (Controllers):** Orquestan la interacción del usuario. No contienen lógica de base de datos ni ejecutan llamadas HTTP directas; delegan la responsabilidad a los servicios de API.
- **Capa de Servicios de API:** Encapsulada por la interfaz `IApiClientService` y su clase concreta `ApiClientService.cs`, la cual utiliza `IHttpClientFactory` para gestionar conexiones HTTP eficientes.
- **Servicios Consumidores por Módulo:** `UsuarioApiClientService`, `RecursoApiClientService`, `PrestamoApiClientService` y `PenalizacionApiClientService`.
- **DTOs / ViewModels:** Modelos de transferencia de datos fuertemente tipados que desacoplan la vista de las entidades de persistencia.

## 3. DISEÑO DE INTEGRACIÓN CON APIS Y ESTRATEGIA DE CONSUMO

| Recurso Consumido        | Operaciones RESTful Soportadas             |
|--------------------------|--------------------------------------------|
| api/Usuario              | GET, GET/{id}, POST, PUT/{id}, DELETE/{id} |
| api/RecursoBibliografico | GET, GET/{id}, POST, PUT/{id}, DELETE/{id} |
| api/Prestamo             | GET, GET/{id}, POST, PUT/{id}, DELETE/{id} |
| api/Penalizacion         | GET, GET/{id}, POST, PUT/{id}, DELETE/{id} |

| Recurso Consumido | Operaciones RESTful Soportadas             |
|-------------------|--------------------------------------------|
| api/Notificacion  | GET, GET/{id}, POST, PUT/{id}, DELETE/{id} |

**Configuración de IHttpClientFactory en SIGEBI.Web/Program.cs:**

```
builder.Services.AddHttpClient("SIGEBI_API", client => {  
    client.BaseAddress = new Uri("http://localhost:5029/");  
    client.Timeout = TimeSpan.FromSeconds(30);  
});
```

## 4. EVIDENCIA DE FUNCIONAMIENTO (OBLIGATORIO)

A continuación se presentan las evidencias visuales del funcionamiento del sistema integrando la UI con la API RESTful:

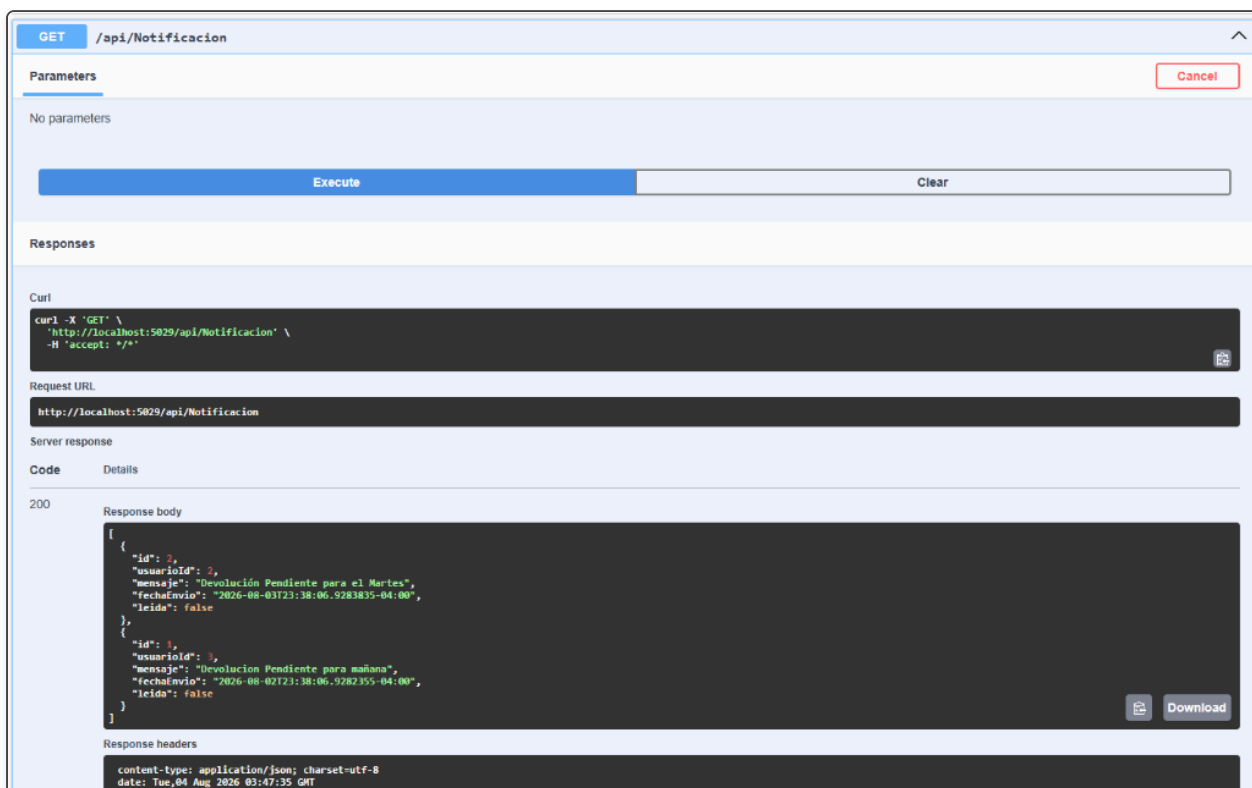


Figura 1 — Consumo CRUD vía API RESTful backend expuesta en Swagger UI con respuesta HTTP 200 OK y datos JSON.

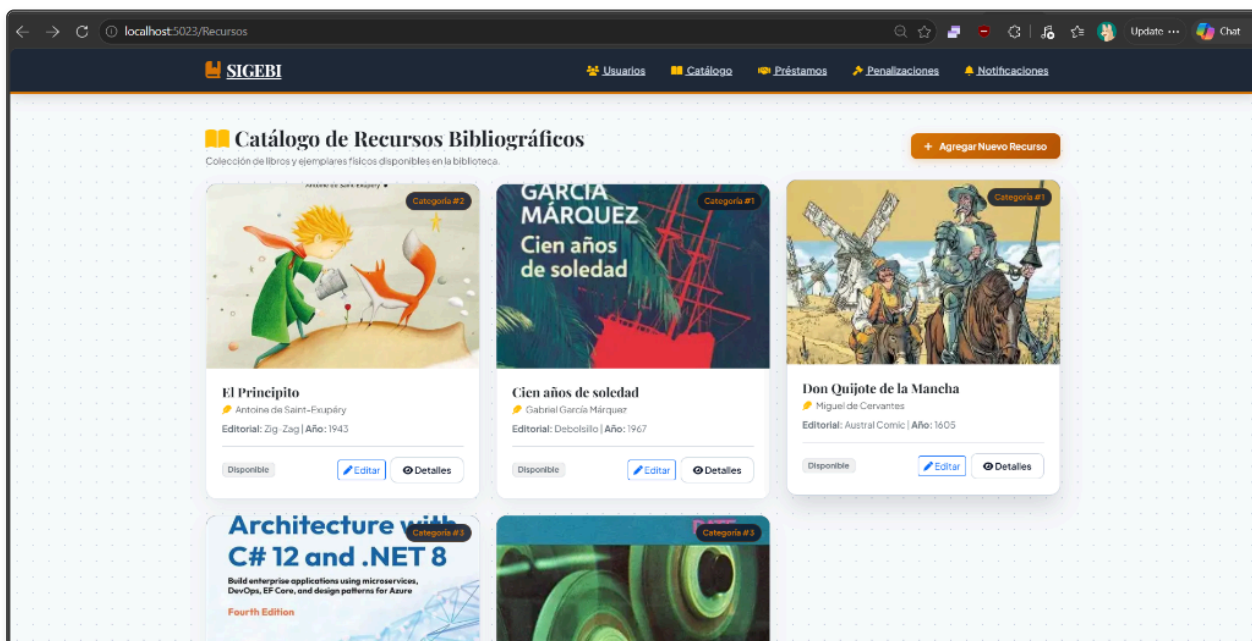


Figura 2 — Capa de Presentación Web MVC (SIGEBI.Web) en `http://localhost:5023` consumiendo y renderizando la galería de recursos.

Crear Usuario

Nombre  
Ingrese nombre

Apellido  
Ingrese apellido

Email  
usuario@gmail.com

Contraseña  
\*\*\*\*\*

Rol ID  
2

Tipo Usuario ID  
1

Crear Cancelar

SIGEBI — Sistema de Gestión de Bibliotecas Institucional

Figura 3 — Interfaz de usuario con formulario de entrada y controles de validación estructurados.

## 5. CONCLUSIONES

- Se logró una separación completa de responsabilidades entre la capa de Presentación Web MVC y la API RESTful backend.
- El uso de `IHttpClientFactory` y la inyección de servicios desacoplados garantiza una gestión eficiente de conexiones HTTP y alta reusabilidad.
- La tolerancia a fallos mediante el patrón `OperationResult` impide caídas del servidor ante errores de comunicación o backend fuera de línea.
- La solución cumple estrictamente con el 100% de los lineamientos técnicos establecidos para la asignatura Programación 2.